8-2 · DAG 이해 및 유즈케이스 연구 · 정리·회고 0.5H

정리·회고

DAG 작성 규칙을 한 장으로, 8-3 설계안을 확정

수도권ICT 이노베이션스퀘어

데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

30분 뒤에는

① DAG 선언·순서·재시도·af 명령을 한 장으로 정리한다

② 자주 겪는 오류 네 가지의 원인과 해결을 안다

③ pipeline_skeleton.py 가 남아 있는지 보고 8-3 준비를 마친다

④ 오늘의 체크리스트를 스스로 확인한다

논리적 날짜·paused·catchup — 오늘 헷갈렸던 것은 여기서 털고 갑니다

오늘의 요약

DAG 파일 안 — 선언·태스크·순서·재시도

하려는 일	코드
DAG 선언	with DAG(dag_id=, start_date=, schedule=, catchup=False) as dag:
파이썬 태스크	PythonOperator(task_id=, python_callable=)
순서 · 병렬 · 합류	t1 >> t2 · t1 >> [t2, t3] · [t2, t3] >> t4
재시도	default_args={"retries": 2, "retry_delay": timedelta(minutes=1)}

학습 DAG 는 catchup=False 를 반드시 명시 — start_date 가 과거면 소급 실행이 쏟아진다

오늘의 요약

af 명령 일곱 — 새 파일 읽기부터 다시 돌리기까지

하려는 일	명령
새 파일 바로 읽기	af dags reserialize
단위 실행	af tasks test <dag> <task> <날짜>
활성화	af dags unpause <dag>
수동 실행	af dags trigger <dag>
실패한 곳부터 다시	af tasks clear <dag> -t <task> -d -y
파싱 오류	af dags list-import-errors
의존성 보기	af dags show <dag>

af = docker compose exec airflow-scheduler airflow — 새 터미널이면 함수를 다시 만든다

오늘의 체크리스트

여덟 개를 스스로 확인해 보세요

1. DAG·Task·Operator 관계를 그림으로, 나누는 기준을 한 문장으로
2. hello_dag 가 웹 UI 목록에 나타나는 것을 확인했다
3. 구문 오류를 일부러 내 list-import-errors 로 잡아 봤다
4. af tasks test 로 단위 실행하고, 새 파일을 reserialize 로 바로 읽혔다
5. unpause → trigger 로 전체를 돌리고 Graph 화살표와 대조했다
6. 중간 태스크를 실패시켜 upstream_failed 를 보고 tasks clear 로 재실행했다
7. 병렬·합류 DAG 와 순환 거부를 직접 봤다
8. 멱등 여부를 적은 분해안과 pipeline_skeleton 골격을 만들었다

자주 겪는 오류

증상으로 찾는 원인과 해결

증상	원인	해결
저장했는데 목록에 안 나타난다	dags/ 밖 · 구문 오류 · 새 파일 주기 5분	list-import-errors → reserialize
켜자마자 과거 실행이 잔뜩 쏟아진다	start_date 과거 + catchup=True(미지정)	catchup=False 명시 · 쌓인 실행은 Grid 에서 삭제
trigger 는 됐는데 태스크가 계속 회색	DAG 가 paused — 새 DAG 의 기본값	웹 UI 토글 또는 dags unpause <dag_id>
DAG 파일 하나로 전체가 느려진다	최상위에 무거운 코드 — 파싱마다 실행	무거운 일은 태스크 함수 안에

DAG 파일은 스케줄러가 주기적으로 통째로 파싱한다 — 최상위에 DB 접속·API 호출을 두지 않는다

예상 질문

자주 나오는 질문 네 가지

1. **태스크끼리 값을 주고받으려면?**
   XCom — 다만 큰 데이터는 넘기지 말고 저장소 경로만 넘기는 것이 원칙

2. **태스크가 몇 개까지 괜찮나요?**
   수십 개까지는 흔하다 — 수백이 되면 DAG 를 나누거나 동적 생성을 검토

3. **schedule=None 이면 자동 실행이 안 되는데 왜 쓰나요?**
   다른 DAG 가 트리거하거나 사람이 돌리는 작업에 — 학습에도 편하다

4. **TaskFlow API(@task)는 안 배우나요?**
   더 간결한 최신 문법 — 오늘은 Operator 로 구조를 먼저, 여유가 되면 심화 과제로

8-3 준비

다음 차시를 위한 준비 — 골격 파일 하나

1. **cd ~/airflow-lab · ls dags/**
   pipeline_skeleton.py 가 보이면 8-3 준비 완료

2. **7주차 컨테이너 ollama · qdrant · rag-minio**
   8-3 실습점검에서 다시 켠다 — 지금은 그대로

3. **lecture_kb 10 조각**
   8-3 실습점검 2단계에서 7-5 끝 상태인지 본다

준비 완료

골격 파일이 남아 있다

심화 과제(선택): 크론 표기로 다음 실행 시각 보기 · TaskFlow API 로 다시 쓰기 · XCom 으로 건수 넘기기

다음 회차 예고

8-3 · 수집→Lake→Spark→VectorDB 흐름 자동화

01
이론 1H
파이프라인
자동화 설계

02
실습 1.5H
수집→Lake→변환
태스크 구현

03
실습 1H
임베딩→VectorDB
전체 DAG 완성

04
정리 0.5H
실행 결과 점검
컬렉션 상태 확인

준비: 7주차 컨테이너 셋과 lecture_kb 10 조각 · 오늘 확정한 태스크 분해안 · OpenAI 키는 OpenAI 경로만

8-2 · 정리·회고 0.5H 마무리

다음 회차는 8-3 흐름 자동화

손으로 돌리던 수집→Lake→검증→임베딩→VectorDB 적재를 DAG 하나로 잇고, 트리거 한 번으로 끝까지 돌립니다

- ✓ **7주차 컨테이너를 Airflow 네트워크에 붙이기**
- ✓ **7주차 build_kb.py 를 그대로 태스크로**
- ✓ **태스크 간 데이터는 XCom 아닌 스토리지 경로로**
- ✓ **SparkSubmitOperator 의 구조**

준비할 것

분해안 문서 · pipeline_skeleton.py